



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Optimizing LLM inference - A Hardware/Infrastructure perspective

**Presented by Vansh Nagpal, Anshul Batish, Shrey
Kohli, Pranav Gupta**

04/02/2026



Agenda

1. WaferLLM
2. Rearchitecting Datacenter Lifecycle for AI
3. Insights into DeepSeek-V3 (Optional)
4. Conclusion
5. Q&A



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

WaferLLM

Large Language Model Inference at Wafer Scale

Congjie He, Yeqi Huang, and Pei Mu, University of Edinburgh; Ziming Miao, Jilong Xue, Lingxiao Ma, and Fan Yang, Microsoft Research; Luo Mai, University of Edinburgh

Background – LLM Inference Phases

Prefill Phase: Process input tokens (prompt) and generates initial KV cache

Dominated by General Matrix Multiply (GEMM) Operations

Compute-bound, often dictating the Time to First Token (TTFT)

Highly Parallelizable

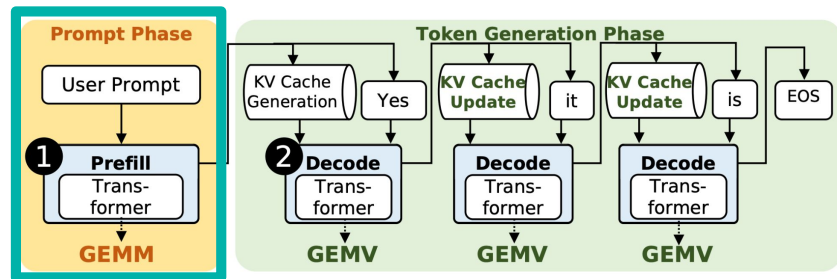


Figure 1: Key components in LLM inference

Background – LLM Inference Phases

Decode Phase: Generates tokens one by one in an autoregressive manner

Decoding driven by General Matrix-Vector Multiplication (GEMV) operations

LLM Inference is memory bandwidth-bound

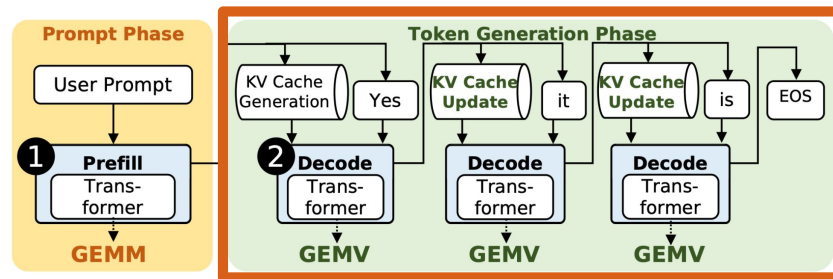


Figure 1: Key components in LLM inference

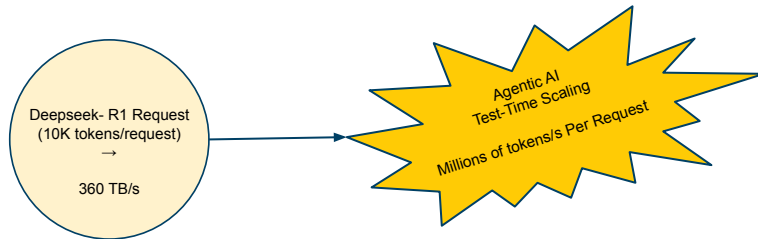
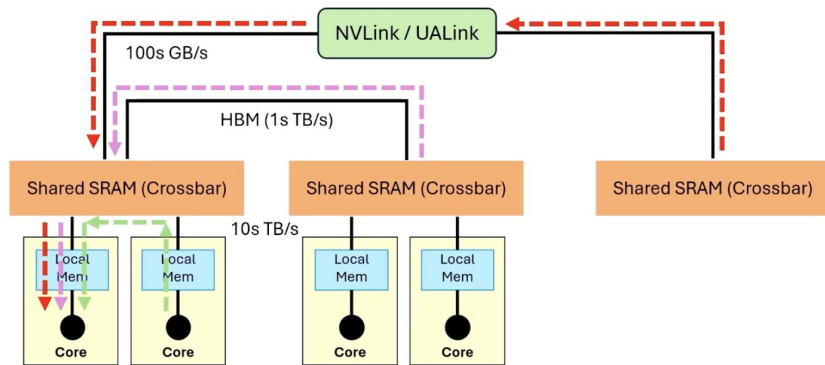
Status Quo and Scaling Problem

Today's systems rely on **off-chip** solutions

Problem:

Off-chip interconnected based system constrained by communication latency and high memory bandwidth (e.g., **HBM** or NVLink)

GPUs constantly fetches model weights from off-chip memory → Idle time



A New Hope: Wafer-Scale Integration

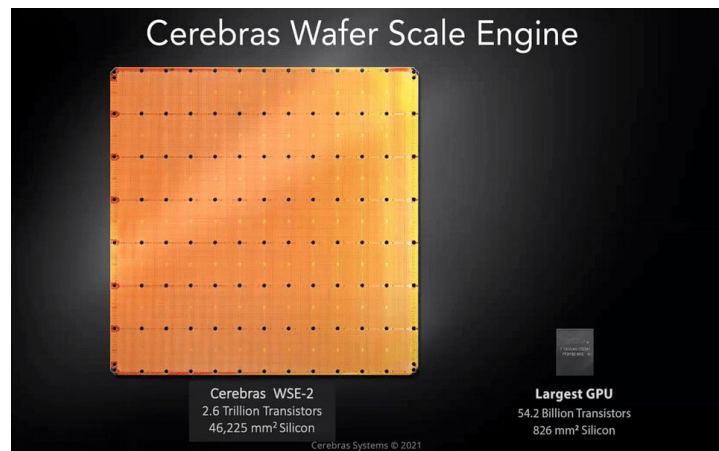
Wafer-scale integration (WSI): integrates entire silicon wafers into a single "super-chip"

Mesh Network-on-Chip (NoC): Mesh-based memory architecture where cores communicate primarily with their immediate neighbors

Cerebras WS-2 (April 2021)

- 2.6 million transistors (100x more than GPUs)
- 850,000 AI-optimized cores
- 40 GB on-chip memory (1,000x more than GPUs)
- 22 PB/s memory bandwidth (7,000x higher than GPUs)

Efficiency: 100x better power efficiency per bit than conventional PCB-based chip-to-chip interconnections (e.g, NVLink)





Challenges for Wafer-Scale LLM inference

Shifting LLM inference to distributed, non-uniform memory architecture on single chip

Scaling LLM with off-chip network research

Network Topology:

3D-Torus, Clos

System: Megatron-LM

Multi-Dimensional

Parallelism: TP, PP, DP, EP

Communication

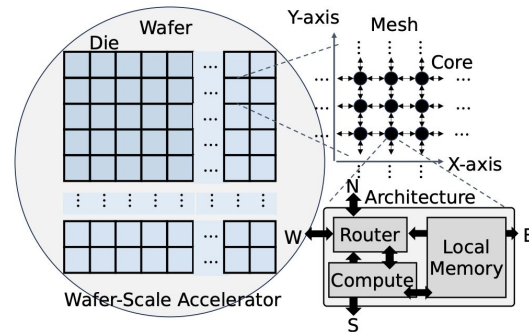
Operator: Ring allreduce, All-to-all MOE

LLM with on-chip shared memory research

Operator: FlashAttention, PageAttention

Compiler: Ladder

Wafer-scale AI software largely unexplored



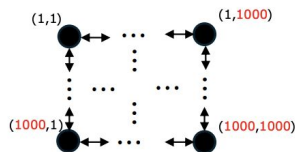
(b) Massive-Sacle, Mesh-Based Memory



PLMR Device Model

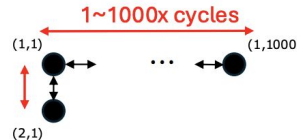
What challenges arise when designing software for wafer-scale hardware?

1. Million-scale Parallelism (PLMR)



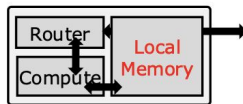
From hundreds of parallelism in a crossbar to millions of parallelism in a mesh

2. Highly non-uniform access Latency (PLMR)



From shared memory and small NUMA to large-scale non-uniform memory

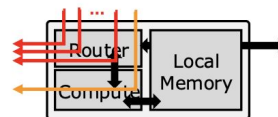
3. Constrained local Memory (PLMR)



100s KB – 1s MB

From coarse-grained tile pipeline to fine-grained tile pipeline

4. Constrained Routing resources (PLMR)



Only support 10s routing entries

From centralized routing to decentralised NoC routing

→ Routing on NoC

→ Routing on Compute Engine



WaferLLM

Contributions:

- New prefill parallelism strategies
- New decode parallelism strategies
- New KV-cache algorithm – Shift-based update
- New GEMM algorithm - MeshGEMM
- New GEMV algorithm - MeshGEMV

Goal: Provide the first LLM inference system specifically designed for wafer-scale accelerators

Prefill Parallelism

Design Fine-Grained Partitioning For Million-Core Parallelism

- Partition two dimensions of the input activation and weight matrices along X- and Y-axes of cores

Designing PLMR-compliant Distributed GEMM

Transposed Distributed GEMM To Avoid Matrix Transpose

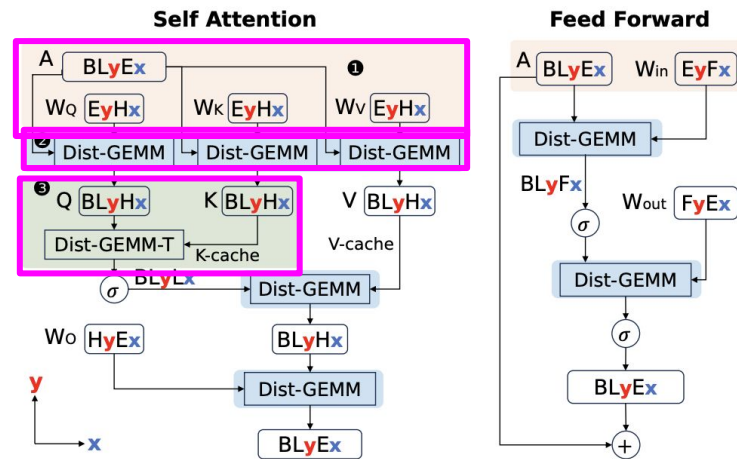


Figure 3: Prefill parallelism plan. $E_x F_y$ represents a matrix of shape EF , where the E dimension is partitioned along the x -axis of cores, and F along the y -axis of cores on a mesh.

Decode Parallelism

Design Fine-Grained Replication To Enable Parallelism at Minimal Communication Cost

- Partition two dimensions of the input activation and weight matrices along X- and Y-axes of cores

Designing PLMR-compliant Distributed GEMV

Pro-optimize Model Weight Placement To Avoid Matrix Transpose

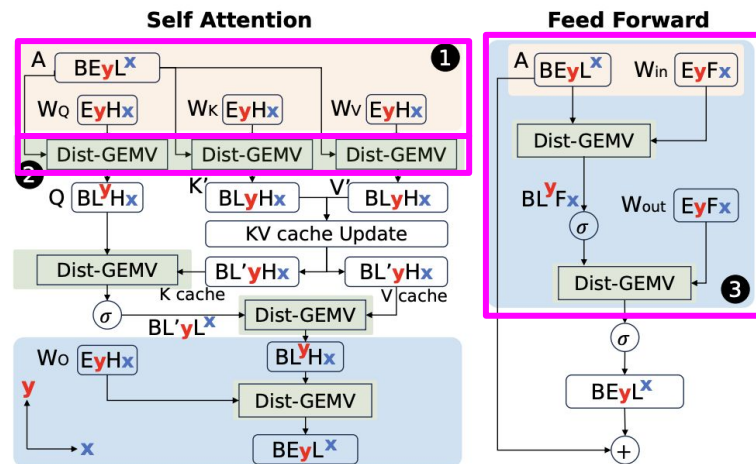
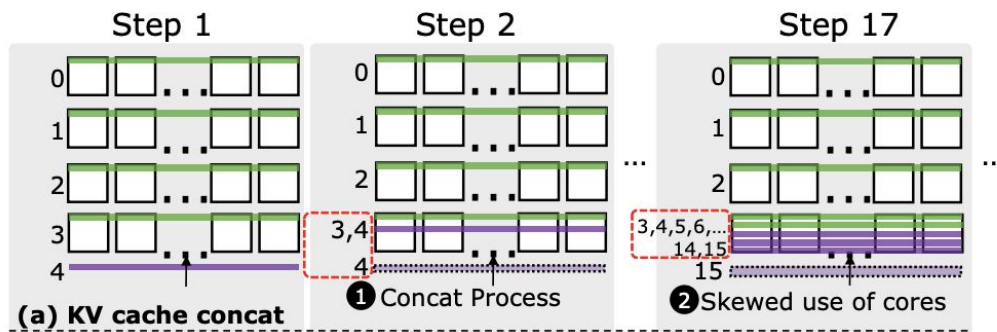


Figure 4: Decode parallelism plan. E^yF_x indicates the E dimension is replicated along the y -axis, and F is partitioned along the x -axis.



Shift-based KV cache management

Problem: Existing
concatentate-based
management causes
skewed core utilization



Shift-based KV cache management

Solution: Shift-based management for balanced core utilization

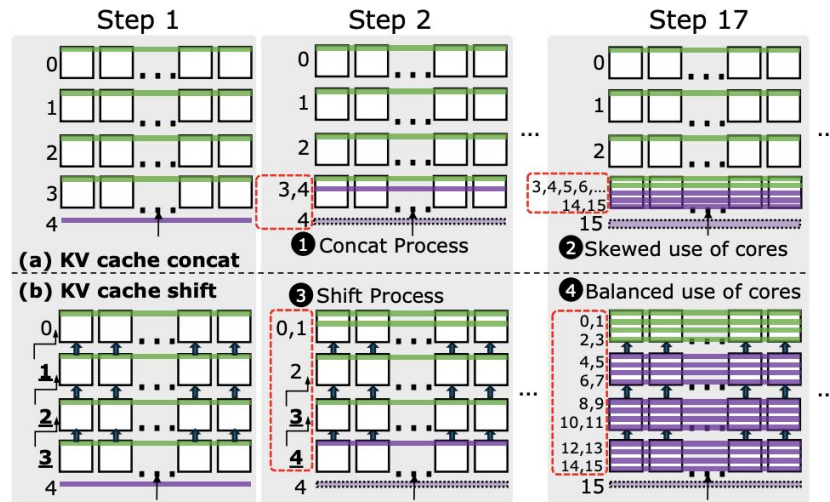


Figure 5: KV cache concatenation vs. KV cache shift

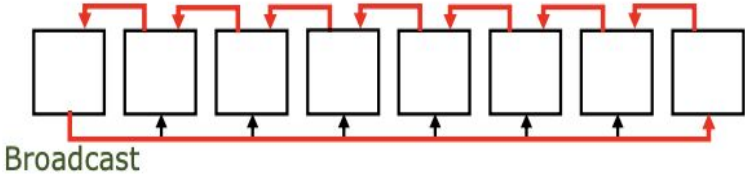
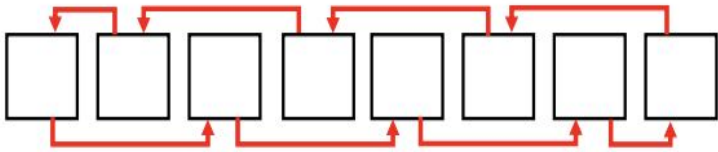


Making Matrix Computation PLMR-Compliant

- **Prefill** stage is dominated by **GEMM** operations
- **Decode** stage is dominated by **GEMV** operations
- Existing distributed computation algorithms **break at least one** of the PLMR requirements

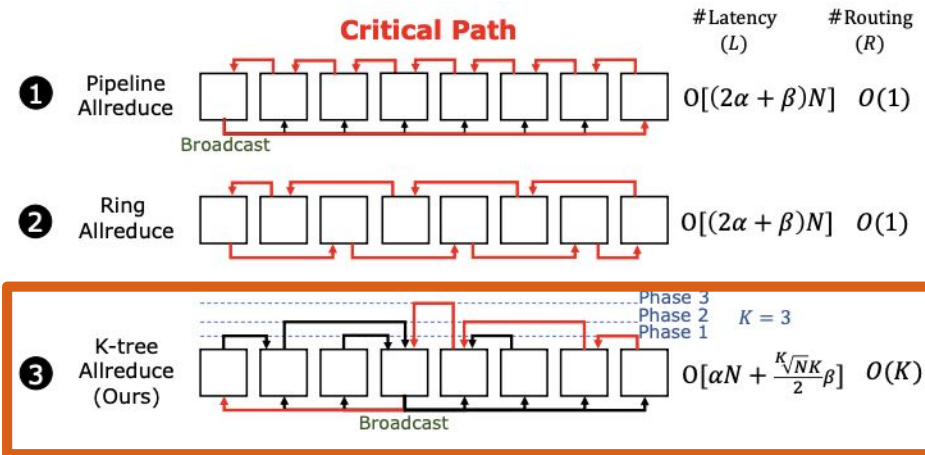


Shortcomings of Existing GEMV Implementations

			#Latency (L)	#Routing (R)	
1	Pipeline Allreduce		$O[(2\alpha + \beta)N]$	$O(1)$	Completely sequential reduce chain (linear latency $2N$ hops)
2	Ring Allreduce		$O[(2\alpha + \beta)N]$	$O(1)$	Overlaps communication Still fundamentally linear latency

Proposed Approach: MeshGEMV

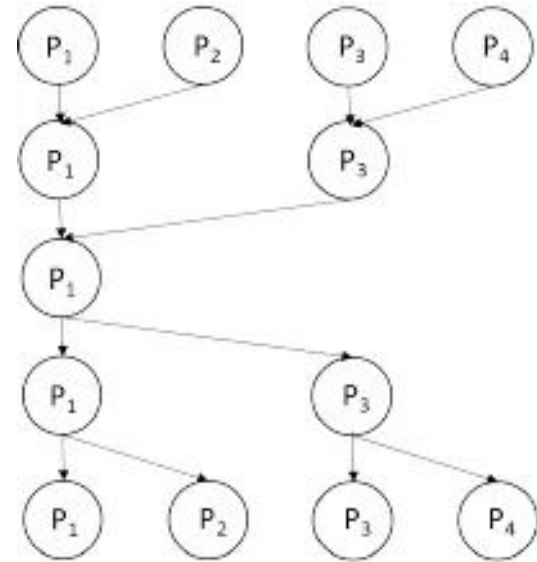
- **K-Tree Allreduce**: Partition cores into groups of K
- Each groups reduces in parallel
- Recursively reduce across groups
- Balanced K-ary tree leads to lower latency but **higher routing complexity** (Tradeoff)
- Broadcast result back to all cores





MeshGEMV Example, $K = 2$

- Each core computes its partial sum
- Over $\log_2(K)$ rounds, the final sum is aggregated in to Core 1
- Core 1 broadcasts it back





MeshGEMV General Algorithm

1. $A \times B$ (A is a vector, B is a matrix); shard B onto $N \times N$ cores and shard A in N pieces and replicate across rows
2. Aggregate and reduce within rows then across leaders (each row and the first column is a K -ary tree)
3. Broadcast result back in reverse order

End-End Inference Eval

- Metric used: Throughput per Request (TPR = 1/TPOT)
- Models tested: LLaMA3-8B, LLaMA2-13B, CodeLLaMA-34B, QWen2-72B
- vs. T10 (SOTA distributed on-chip compiler): **100–200× speedup** on average
- vs. Ladder (SOTA shared-memory compiler): **200–400× speedup** on average
- vs. SGLang on single A100: **30–40× faster** end-to-end
- vs. SGLang on optimal 8-GPU A100 cluster: **10–20× faster, 2.5× more energy-efficient**

Model	Device		Input/Output Sequence Length			
			2048/128	4096/128	2048/2048	4096/4096
LLaMA3-8B	WSE-2	WaferLLM	764.4	604.4	2370.3	2459.0
		T10	4.6	4.5	58.3	94.6
		Ladder	1.2	1.1	7.4	8.7
	A100 GPU (SGLang)	1	34.8	31.1	36.5	78.4
		8	117.2	109.0	128.4	256.1
		2×8	73.7	70.2	79.3	162.5
LLaMA2-13B	WSE-2	WaferLLM	473.9	414	1690.3	1826.0
		T10	2.6	2.5	55.0	58.5
		Ladder	0.7	0.7	4.9	6.1
	A100 GPU (SGLang) ³	1	20.4	17.1	21.1	47.9
		8	79.6	70.5	86.9	172.4

Prefill and Decode TPR

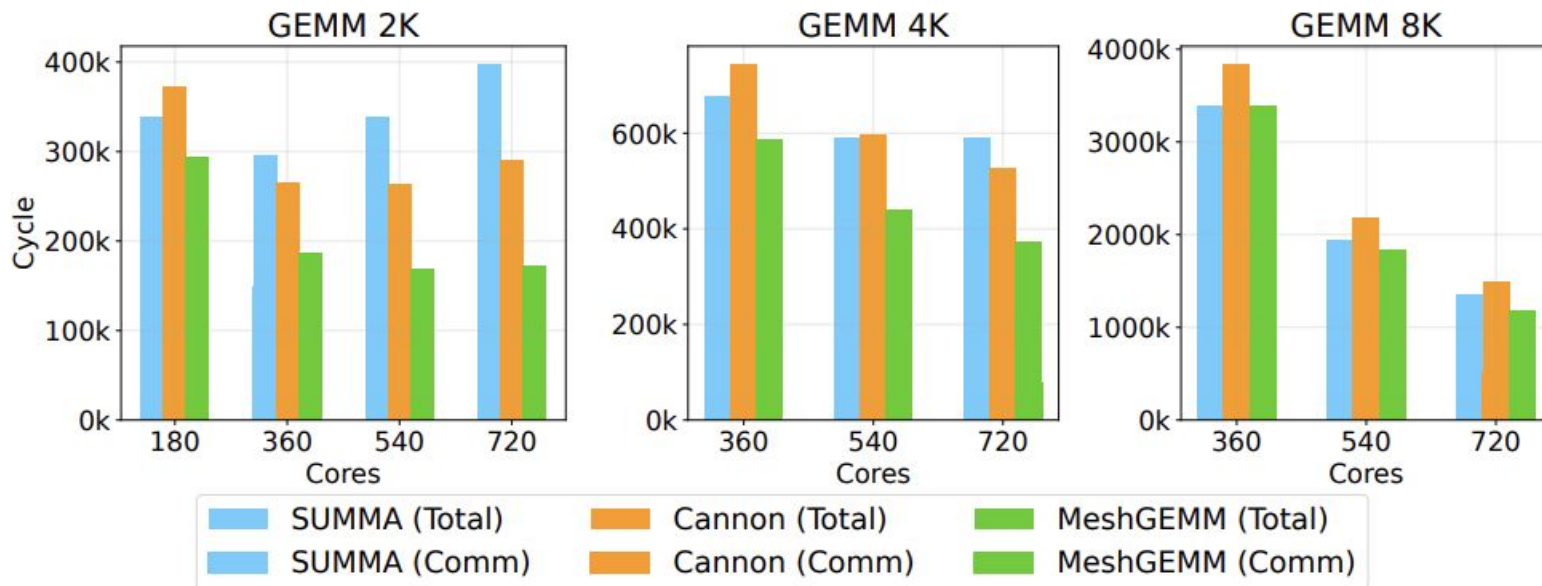
Model	Methods	WSE-2 Cores			# A100 GPUs (SGLang)		
		480×480	600×600	720×720	1	8	2×8
LLaMA3-8B	WaferLLM	20320.6	25037.2	27686.5	13988.3	17361.6	13994.2
	T10	175.0	156.6	132.8			
	Ladder	61.8	42.3	31.3			
LLaMA2-13B	WaferLLM	13685.1	16854.2	17498.3	7805.1	12287.1	/ ³
	T10	121.3	100.6	81.3			
	Ladder	47.3	33.1	24.2			
CodeLLaMA-34B	WaferLLM	5471.4	7540.1	8526	5382.5	7155.5	6409.2
	T10	49.1	46.8	41.2			
	Ladder	30.1	23.1	17.7			
QWen2-72B	WaferLLM	2785.2	3775.5	4421.6	1677.3	3803.8	3750.5
	T10	24.9	23.5	21.5			
	Ladder	16.8	12.8	10.1			

Table 3: Prefill Throughput per Request (TPR)

Model	Methods	WSE-2 Cores			# A100 GPUs (SGLang)		
		420×420	540×540	660×660	1	8	2×8
LLaMA3-8B	WaferLLM	2699.9	2501.5	2243.3	78.9	260.4	164.6
	T10	418.3	339.4	265.1			
	Ladder	14.6	13.1	11.4			
LLaMA2-13B	WaferLLM	2039.2	1899.4	1739.8	48.7	175.8	/ ³
	T10	341.8	270.8	233.7			
	Ladder	11.0	9.9	9.0			
CodeLLaMA-34B	WaferLLM	1450.8	1407.7	1359.2	26.1	100.4	84.5
	T10	278.2	222.4	193.1			
	Ladder	6.1	6.2	5.8			
QWen2-72B	WaferLLM	839.7	824.3	787.1	10.6	51.2	48.7
	T10	168.5	133.0	114.6			
	Ladder	3.2	3.3	3.4			

Table 4: Decode Throughput per Request (TPR)

MeshGEMM Eval



MeshGEMV Eval

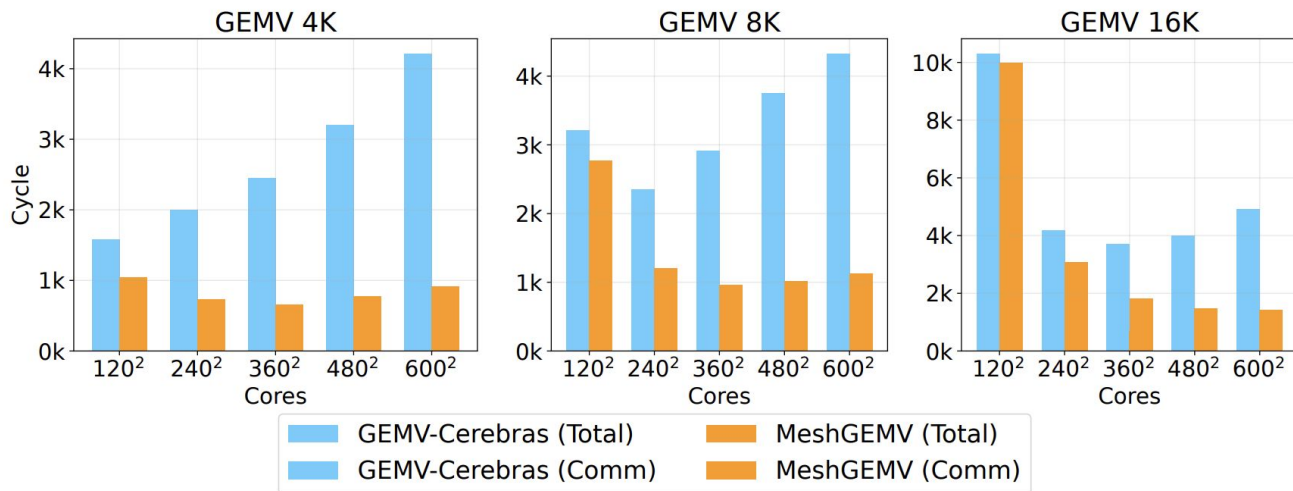
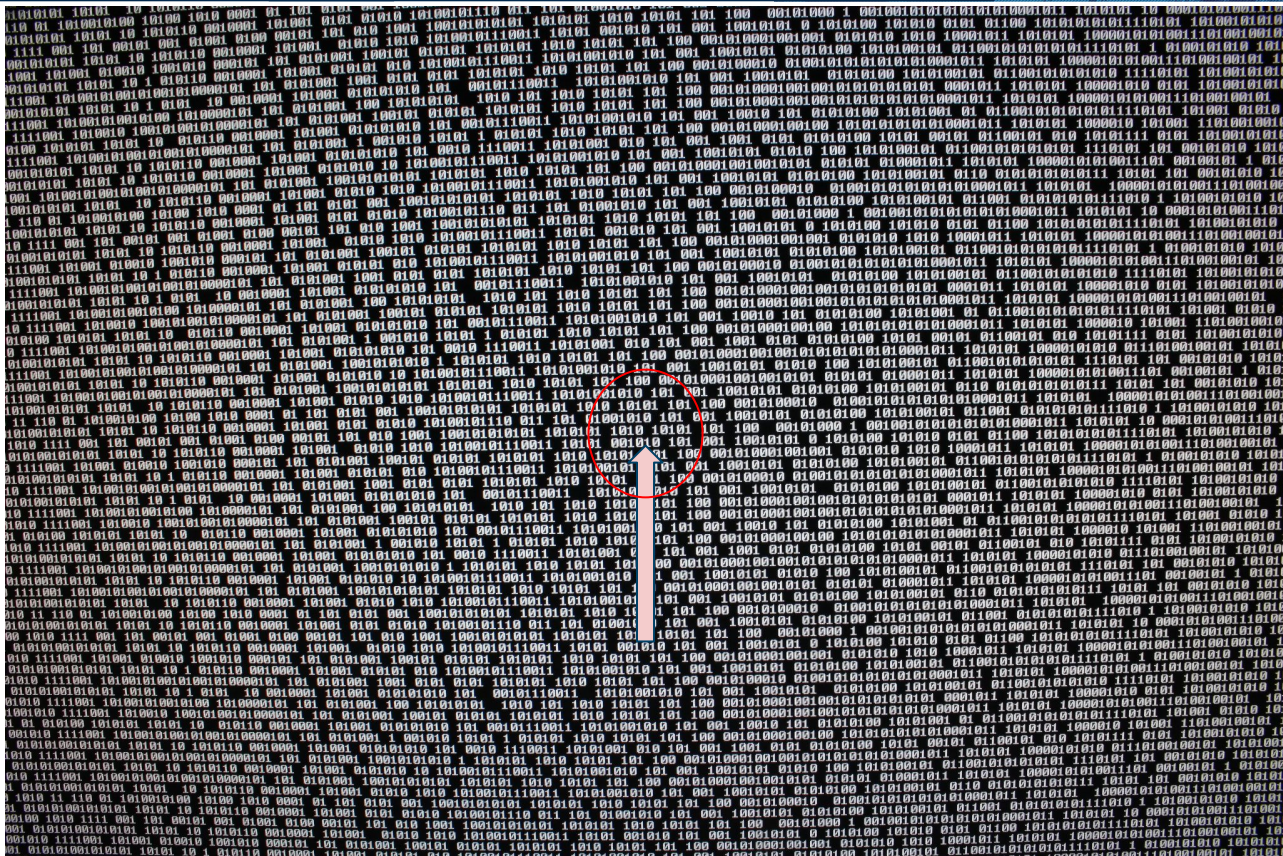


Figure 10: MeshGEMV vs. GEMV-Cerebras





Rearchitecting Datacenter Lifecycle for AI

A TCO-Driven Framework

Jovan Stojkovic, University of Illinois; Chaojie Zhang and Íñigo Goiri, Microsoft Azure Research; Ricardo Bianchini, Microsoft Azure

Introduction

- LLMs are driving massive demand for AI inference infrastructure in cloud datacenters.
 - serving depends on high-end GPUs, which deliver strong performance but also create very high capital, power, and cooling costs.
- Traditional datacenter lifecycle management was designed for CPU-centric general-purpose workloads, so it does not fit AI well.

Their Work. A framework that rethinks and rearchitects the entire lifecycle for AI datacenters, by exploring alternative strategies and identifies the most cost-effective combination

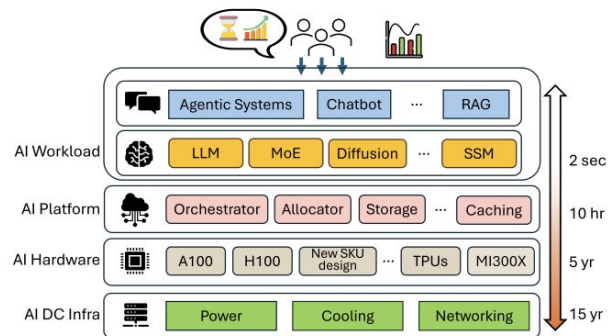


Figure 1. Hosting AI workloads from models to hardware and supporting datacenter infrastructure.

Total Cost of Ownership (TCO)

$$\text{TCO} = \text{CapEx} + \text{OpEx}$$

Their system reduces the TCO by up to 40% over traditional approaches.

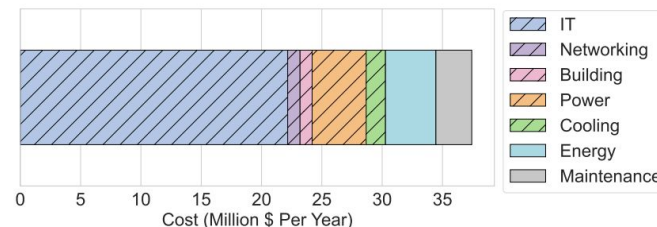


Figure 3. TCO breakdown for a 10MW AI datacenter.

Category	Component	Description	Example Cost (\$)
CapEx	IT	Servers, racks, accelerators, storage	\$375k/server [18]
	Networking	Fabric switches, optics, structured cabling	\$2000/server [59, 85]
	Building	Site preparation, building shell, land, electrical and mechanical base infrastructure	\$0.5/ft ² [29]
	Power	Power infrastructure (Switchgear, transformers, UPS, PDUs, busbars, rack distribution)	\$7.0/W [26, 116]
	Cooling	Cooling infrastructure (chillers, CRAH/CRAC units, pumps, piping, liquid loops, airflow)	\$2.5/W [103, 116]
OpEx	Networking	Port licenses, optics replacement, networking component power	\$600/server [59, 82, 84]
	Energy	IT load scaled by PUE, utility tariffs, demand charges	\$20–40/MWh [35]
	Maintenance	Spares, repairs, monitoring, water/treatment, field-replaceable units, failure-rate	\$5000/server [102]
	Software	Licenses, support contracts	\$200/server

Table 3. TCO components for an example datacenter with DGX H100 [18] servers.

Hosting AI Workloads

- Models have grown *dramatically* in size demanding multi-gpu setups
- AI workload architectures boil down to matrix multiplications so easier to make a unified model compared to general purpose workloads.
- User demand is increasing: projected to grow from \$638B in 2024 to over \$3.68T by 2034.
 - Inference costs > training costs due to continuous large-scale deployment serving millions of queries each day.



$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

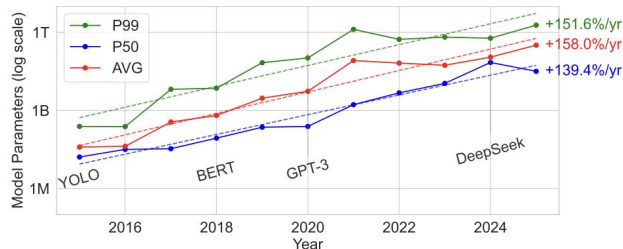


Figure 2. The P50, P99, and average size of the most popular AI models published in the last decade.



Also...infrastructure

Power: rack densities can range from several kilowatts to over 100 kW per rack, uninterruptible power supplies (UPS) and power distribution unit (PDU) topologies

Cooling: advanced cooling solutions such as rear-door heat exchangers, direct-to-chip liquid cooling, and, in ultra-dense deployments, full liquid cooling

Networking: models using tensor, pipeline, or expert parallelism rely on low-latency, high-bandwidth communication between thousands of accelerators

The three stages:

1. **Build**: initial infrastructure setup, including provisioning decisions about power topology, cooling, networking, facility design
2. **IT provisioning**: governs when and how to decommission and upgrade hardware
3. **Operations**: runtime management: workload placement, scheduling, and software optimizations.



Stage	Description	Timeline
<i>Build</i>	Site selection and facility construction	15–30 years
<i>IT provision</i>	IT hardware deployment and upgrades	4–6 years
<i>Operate</i>	Workload scheduling, resource management	Per inference

Table 1. Lifecycle stages for datacenter infrastructure.

Stage	Traditional Approach Characteristics
<i>Build</i>	Hierarchical power distrib; Air cooling; Ethernet network.
<i>IT provision</i>	Fixed per-server lifecycle; New server generations released every 2–3 years; Gradual replacement.
<i>Operate</i>	Services tied to fixed hardware configurations; Instances migrated to new hardware when released; Legacy applications remain on old servers.

Table 2. Overview of the traditional approach to manage the lifecycle of a traditional datacenter for general-purpose serving CPU-based workloads.



TCO-Driven Lifecycle Framework

Modeling Assumptions:

- **Timeline:** 15-year lifecycle (2015–2030), validated till 2025 and forecasted ahead
- **Workload:** LLM inference dominant; starts at 100K RPS with ~15% annual growth
- **AI Models:** Model size scales (linear baseline); smooth transitions; decoder-only transformers (LLaMA-style)
- **Hardware:** Linear growth of FLOPS, Memory Bandwidth, power dissipation and cost
- **Performance Modeling:** Roofline model estimates latency (TTFT, TBT) using compute + memory limits

SLO Constraint: Maintain latency thresholds → 400ms TTFT, 100ms TBT (load increased until SLO violates), derive maximum sustainable goodput

Provisioning Logic: Allocate minimum GPUs required to meet demand within SLO

Lifecycle Evaluation

- Simulate the baseline policy
- See how the AI fleet changes over time as model sizes increase
 - As models grow larger and demand increases, newer GPUs are introduced and the total number of servers increases significantly.
- Monte Carlo Simulations. They do not assume the future is fixed. Instead, they **run the datacenter model many times with different possible futures**—e.g., different demand growth, model sizes, GPU release delays, and costs—to see a **range of possible TCO outcomes** rather than one number.
- Then they **compare different strategies** like upgrading hardware early vs waiting longer.
- By looking at which strategy performs better across many simulated futures, they find **which decisions matter most for long-term cost and where being flexible is most valuable**.

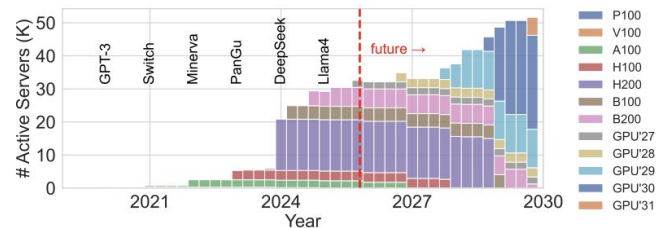


Figure 4. Server count by GPU type over time in an AI fleet following the traditional baseline in Table 2. Includes the release dates of major AI models.

Stage	Description	Timeline
<i>Build</i>	Site selection and facility construction	15–30 years
<i>IT provision</i>	IT hardware deployment and upgrades	4–6 years
<i>Operate</i>	Workload scheduling, resource management	Per inference

Table 1. Lifecycle stages for datacenter infrastructure.

- [illegible]

Table 4. Comparison of power delivery infrastructure designs. Green: good, yellow: moderate, red: poor.

Table 4. Comparison of power delivery infrastructure designs. Green: good, yellow: moderate, red: poor.

Building Efficient AI Infrastructure (Cooling & Networking)

Cooling Innovations: High-density GPU racks (4-8x heat of CPUs) push air cooling to its physical limits.

- **Hybrid Cooling** (75% liquid cold plates, 25% air) is the sweet spot, yielding a 9% reduction in TCO over the full lifecycle.

Networking Strategy:

- Uniform fabrics are expensive and inefficient for AI communication patterns.
- **Optimal Design:** A hierarchical approach using NVLink within servers, InfiniBand within racks, and Ethernet across racks.
- This balance reduces TCO by 6% while meeting strict latency requirements.

Feature		Air	Hybrid	Liquid
CapEx	Complexity	Lower	Medium	Higher
	Energy efficiency	Lower	Medium	Higher
OpEx	Maintenance	Lower	Medium	Higher
	High-dense racks	Lower	Medium	Higher
Other	Noise level	Higher	Medium	Lower

Table 5. Comparison of cooling infrastructure designs.

Feature		Ethernet	InfiniBand	NVLink	Hierarchical
CapEx	Cost	Lower	Medium	Higher	Medium
	Energy	Lower	Medium	Higher	Higher
OpEx	Maintain	Lower	Medium	Higher	Medium
	Bandwidth	Lower	Higher	Higher	Higher
Perf	Latency	Higher	Lower	Lower	Medium

Table 6. Comparison of networking infrastructure designs.



Jovan Stojkovic, Chaojie Zhang, Ñigo Goiri, and Ricardo Bianchini

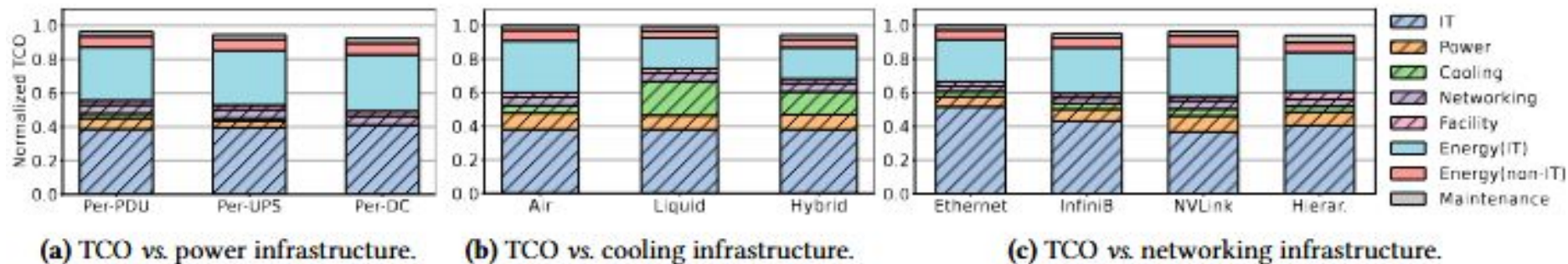


Figure 6. TCO vs. infrastructure designs during the *build* stage.

Provisioning AI Hardware

Ran performance comparisons across diverse models (Llama3, Qwen3, Mamba) on all GPU generations (T4 to H200):

- **Model Size Matters:** Older GPUs remain viable platforms for smaller models; furthermore, the TBT (decode) phase is naturally less sensitive to hardware age due to its lower computational demands. (Fig 8)
- **Sparsity Extends Hardware Life:** Sparse model architectures (Qwen3) degrade less on older hardware compared to dense counterparts (Llama3) while maintaining accuracy, making them a key tool for maximizing legacy hardware utility. (Fig 9)
- **SSMs Offer Hardware Compatibility:** Alternative architectures, specifically state-space models (Mamba), are inherently more hardware-efficient than transformers, showing significantly less performance slowdown when ported to older GPUs (e.g., V100). (Fig 10)

The best hardware choice depends entirely on model: while the H100 is vastly more efficient for large models (e.g., 70B), older GPUs are more economical for smaller models, where the A100 can actually deliver up to 23% better Goodput/Watt/\$ than the H100.

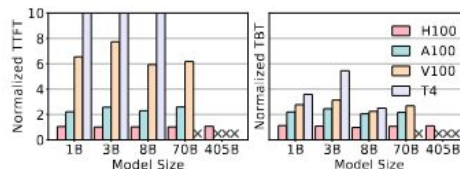


Figure 8. TTFT and TBT latencies for different sizes of Llama-3 LLM across GPU generations normalized to H200.

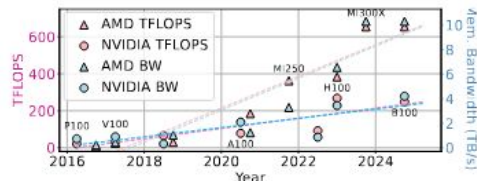


Figure 7. Evolution of AMD and NVIDIA GPUs showing TFLOPS (left axis) and memory bandwidth (right) over time.

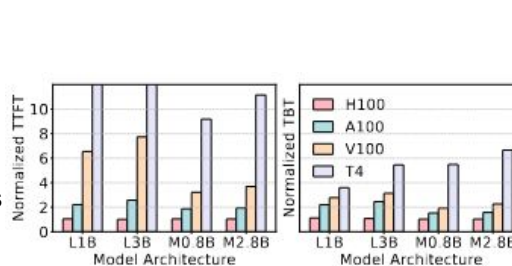


Figure 10. Latencies for transformer (Llama3) and state space (Mamba) models across GPUs normalized to H200.

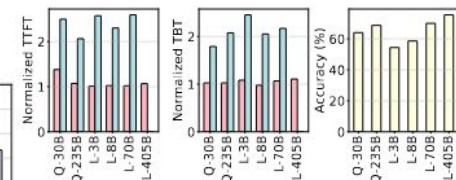


Figure 9. Latencies and accuracies for dense (Llama3) and sparse models (Qwen3) across GPU generations normalized to H200. Pink and blue bars for H100 and A100, respectively.

Rethinking Refresh Policies

The traditional 5-year fixed refresh cycle (for CPUs) is suboptimal for AI workloads.

Dynamic/Flexible Strategies:

- **Newer is Better:** Transitioning from V100 to A100 is justified by massive efficiency gains.
- **Generation Skipping:** If a new GPU offers marginal gains (e.g., B100/B200), it is more cost-effective to extend the life of existing H100s.
- **TCO Impact:** Flexible, workload-aware refresh policies can reduce TCO by 15–20% compared to the baseline.
- Optimal refresh policies to minimize TCO can at times reduce the total server count due to improved GPU efficiency.

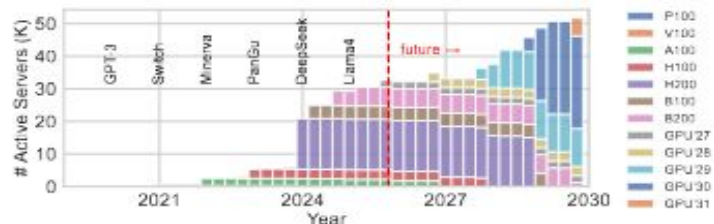


Figure 4. Server count by GPU type over time in an AI fleet following the traditional baseline in Table 2. Includes the release dates of major AI models.

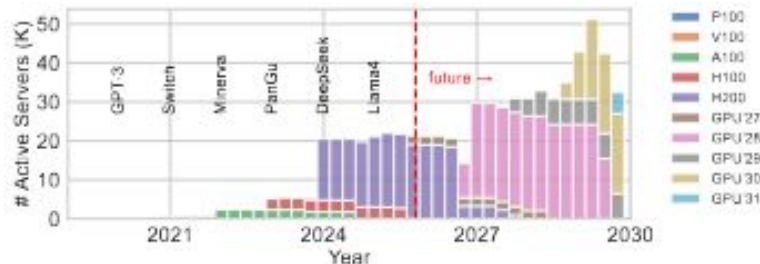


Figure 13. Server count by GPU type over time in an AI fleet following the optimal refresh policy for minimizing TCO.

Operating an AI Datacenter

Traditional vs. AI:

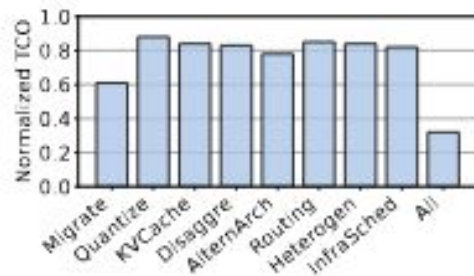
- In traditional datacenters, performance and power efficiency scale linearly with newer hardware, justifying a simple 'migrate to latest' strategy.
- For AI, efficiency is non-linear; the benefit of new hardware varies wildly across different models and use cases.

Key Software Optimization Techniques:

- **Heterogeneity-Aware Scheduling:** Dynamically matches specific workload characteristics to the hardware generation that serves them most efficiently.
- **Infrastructure-Aware Scheduling:** Uses software controls (like dynamic sharding) to optimize for physical datacenter constraints such as power and cooling limits.
- **Phase Disaggregation:** Splits distinct model phases (e.g., prefill vs. decode) to run on the most suitable hardware generations.

The TCO:

- Individual software strategies reduce TCO by 12–39% compared to the baseline.
- **Smoothed Model Migration** provides the largest gains by reducing compute load spikes after new model releases.
- **Cumulative Impact:** Combining all operational optimizations yields a massive 60%+ TCO reduction in this domain.



(a) TCO vs. software optimization.

Optimization Technique

Smooth Model Migration [45, 80]

Model Quantization [63, 118]

KV-Cache Management [27, 83]

Disaggregation [83, 89, 105, 120, 122]

Alternative Architectures

Model Routing [22, 52]

Heterogeneity-Aware Scheduling [54, 65]

Infrastructure-Aware Scheduling [99, 100]

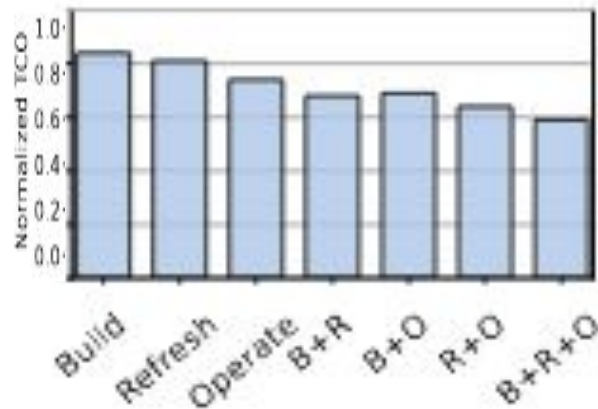
Cross-Stage Optimizations

Existing Cross-Stage Optimizations:

- **IT Provisioning → Build:** Using high-density power and liquid cooling raises initial costs but extends the usable lifetime of the facility.
- **Operate → Provisioning:** Older GPUs are repurposed for decode phases or small models, preventing premature hardware retirement.
- **Build → Operate:** Infrastructure-aware scheduling uses software controls to enable oversubscription and sustains efficiency without new construction.

The Bottom Line (TCO Reductions):

- **Individual Stage Optimization:** Yields 20–30% savings.
- **Combined Build & Refresh:** Pushes savings beyond 35%.
- **Holistic Lifecycle Management:** Achieves over 40% total TCO reduction



(b) TCO vs. stages.

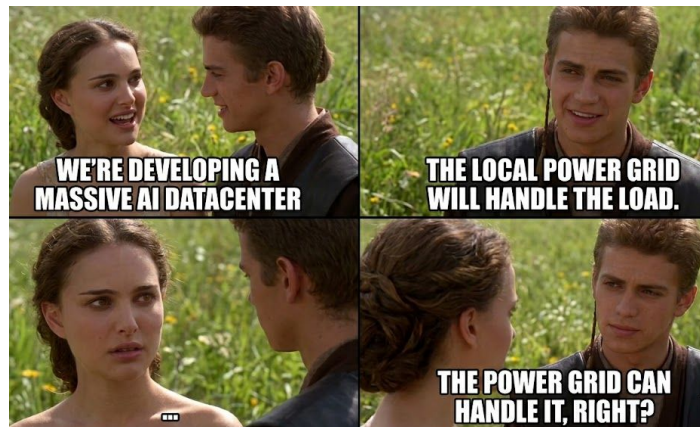
Conclusion

The Shift in Infrastructure Paradigm

- **Inadequacy of Legacy Systems:** Traditional datacenter management designed for general-purpose CPUs is fundamentally mismatched with the power, cooling, and memory demands of modern LLMs.
- **The Wafer-Scale Innovation:** Wafer-scale integration shifts the architecture from shared-memory GPU clusters to distributed, mesh-based on-chip memory with 7,000x higher bandwidth.

Economic & Performance Efficiency

- **Holistic TCO Optimization:** Reaching the full potential of AI infrastructure requires cross-stage coordination (Build + Refresh + Operate), potentially yielding up to a 40% reduction in total cost.
- **Orders-of-Magnitude Gains:** Systems like WaferLLM deliver 10-20x end-to-end speedups and 2.5x better energy efficiency compared to state-of-the-art multi-GPU systems.





Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures

A Practical Implementation

Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma

DeepSeek-V3 Technical Innovations

DeepSeek-V3 uses hardware-software co-design to train a massive 671 billion parameter model using only 2,048 GPUs. This approach focuses on making the math of the AI fit the physical limits of the hardware.

Technical Contributions:

1. Multi-head Latent Attention (MLA) This is a way to shrink the memory needed for an AI to remember a conversation. Usually, as you talk more, the AI's memory cache (KV cache) gets so big it slows down. MLA compresses this memory into a tiny latent vector. **(KV Cache Management)**

- **Improvement:** It uses about 7x less memory than other top models. This lets the AI handle much longer conversations without needing expensive hardware upgrades.

2. Node-Limited MoE Routing The model is a Mixture of Experts (MoE), meaning it only turns on a small part of its brain (37 billion parameters) for any single task. DeepSeek created a strategy where these active experts are forced to stay on the same physical server node. **(Alternate Architectures)**

- **Improvement:** This stops data from having to travel across slow cables between different servers. It keeps the AI fast even when it is performing complex reasoning.

3. FP8 Mixed-Precision Training Most AI training uses 16-bit or 32-bit numbers for math. DeepSeek-V3 is the first major open-source model to use 8-bit (FP8) numbers for the entire training process. **(Model Quantization)**

- **Improvement:** This cuts the memory and energy needed for training in half. Because they used special techniques to keep it accurate, the model lost less than 0.25% of its performance compared to using high-precision math.

4. Multi-Token Prediction (MTP) Instead of predicting just the next word, the model is trained to guess the next few words at the same time. **(Alternate Architectures)**

- **Improvement:** It acts like a fast-thinking assistant that drafts multiple words at once. This makes the AI generate text 1.8x faster because it can check several words in a single step.



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN



Questions